



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

CS23432 - SOFTWARE CONSTRUCTION

Laboratory Observation Note Book

Name : NEIL DANIEL A

Year / Branch / Section : II / CSE

Register No. : 2116240701356

Semester : 4

Academic Year : 2025-2026

Ex No	List of Experiments
1	Study of Azure DevOps
2	Designing Project using AGILE-SCRUM Methodology.
3	Agile Planning
4	User stories – Creation
5	Architecture Diagram Using AZURE
6	Designing Usecase and Class Diagram
7	Designing Interaction Diagrams
8	Design Interface
9	Implementation – Design a Web Page based on Scrum Methodology
10	Testing using Azure.
11	Deployment

Requirements	
Hardware	Intel i3, CPU @ 1.20GHz 1.19 GHz, 4 GB RAM, 32 Bit Operating System
Software	StarUML , Azure

LAB PLAN
CS23432-SOFTWARE ENGINEERING LAB

Ex No	Date	Topic	Page No	Signature
1		Study of Azure DevOps		
2		Writing Problem Statement		
3		Designing Project using AGILE-SCRUM Methodology by using Azure.		
4		Agile Planning		
5		User stories – Creation		
6		Architecture Diagram Using AZURE		
7		Designing Use Case Diagram using StarUML		
8		Designing Activity Diagrams using StarUML		
9		Designing Sequence Diagrams using StarUML		
10		Design Class Diagram		
10		Design User Interface		
11		Implementation – Design a Web Page based on Scrum Methodology		
12		Testing		
13		Deployment		

Study of Azure DevOps

AIM:

To study how to create an agile project in Azure DevOps environment.

STUDY:

Azure DevOps is a cloud-based platform by Microsoft that provides tools for DevOps practices, including CI/CD pipelines, version control, agile planning, testing, and monitoring. It supports teams in automating software development and deployment.

1. Understanding Azure DevOps

Azure DevOps consists of five key services:

1.1 Azure Repos (Version Control)

Supports Git repositories and Team Foundation Version Control (TFVC).

Provides features like branching, pull requests, and code reviews.

1.2 Azure Pipelines (CI/CD)

Automates build, test, and deployment processes.

Supports multi-platform builds (Windows, Linux, macOS).

Works with Docker, Kubernetes, Terraform, and cloud providers (Azure, AWS, GCP).

1.3 Azure Boards (Agile Project Management)

Manages work using Kanban boards, Scrum boards, and dashboards.

Tracks user stories, tasks, bugs, sprints, and releases.

1.4 Azure Test Plans (Testing)

Provides manual, exploratory, and automated testing.

Supports test case management and tracking.

1.5 Azure Artifacts (Package Management)

Stores and manages NuGet, npm, Maven, and Python packages.

Enables versioning and secure access to dependencies.

Getting Started with Azure DevOps

Step 1: Create an Azure DevOps Account

Visit Azure DevOps.

Sign in with a Microsoft Account.

Create an Organization and a Project.

Step 2: Set Up a Repository (Azure Repos)

Navigate to Repos.

Choose Git or TFVC for version control.

Clone the repository and push your code.

Step 3: Configure a CI/CD Pipeline (Azure Pipelines)

Go to Pipelines → New Pipeline.

Select a source code repository (Azure Repos, GitHub, etc.)

Define the pipeline using YAML or the Classic Editor. Run the pipeline to build and deploy the application.

Step 4: Manage Work with Azure Boards

Navigate to Boards.

Create work items, user stories, and tasks.

Organize sprints and track progress.

Step 5: Implement Testing (Azure Test Plans)

Go to Test Plans.

Create and run test cases

View test results and track bugs.

Result:

The study was successfully completed.

EX NO: 2

PROBLEM STATEMENT

AIM:

To prepare PROBLEM STATEMENT for our project.

Problem Statement:

CitizenScheme - Hub

Government scheme information is often stored in large databases and multiple government portals, making it difficult for citizens to quickly identify relevant schemes and benefits. Traditional search systems are inefficient and require users to manually browse through extensive records to find suitable information. The proposed system, **CitizenScheme Hub**, provides a centralized platform that retrieves relevant government scheme details from a database based on user queries and displays accurate and concise results in an easy-to-understand format.

Result:

The problem statement was written successfully.

AGILE PLANNING

AIM:

To prepare the AGILE PLANNING

THEORY:

Agile planning is a part of the Agile methodology, which is a project management style with an incremental, iterative approach. Instead of using an in-depth plan from the start of the project—which is typically product-related—Agile leaves room for requirement changes throughout and relies on constant feedback from end users.

With Agile planning, a project is broken down into smaller, more manageable tasks with the ultimate goal of having a defined image of a project's vision. Agile planning involves looking at different aspects of a project's tasks and how they'll be achieved, for example:

- Roadmaps to guide a product's release ad schedule
- Sprints to work on one specific group of tasks at a time
- A feedback plan to allow teams to stay flexible and easily adapt to change

User stories, or the tasks in a project, capture user requirements from the end user's perspective Essentially, with Agile planning, a team would decide on a set of user stories to action at any given time, using them as a guide to implement new features or functionalities in a tool. Looking at tasks as user stories is a helpful way to imagine how a customer may use a feature and helps teams prioritize work and focus on delivering value first.

Steps in Agile planning process

1. Define vision
2. Set clear expectations on goals
3. Define and break down the product roadmap
4. Create tasks based on user stories
5. Populate product backlog
6. Plan iterations and estimate effort
7. Conduct daily stand-ups
8. Monitor and adapt

RESULT:

Thus the Agile plan was completed successfully.

Aim:

To create User Stories

THEORY

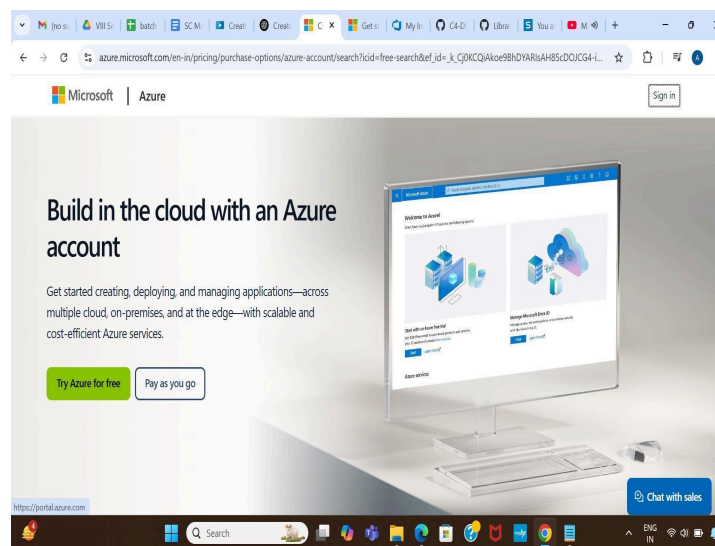
A user story is an informal, general explanation of a software feature written from the perspective of the end user. Its purpose is to articulate how a software feature will provide value to the customer.

User story template

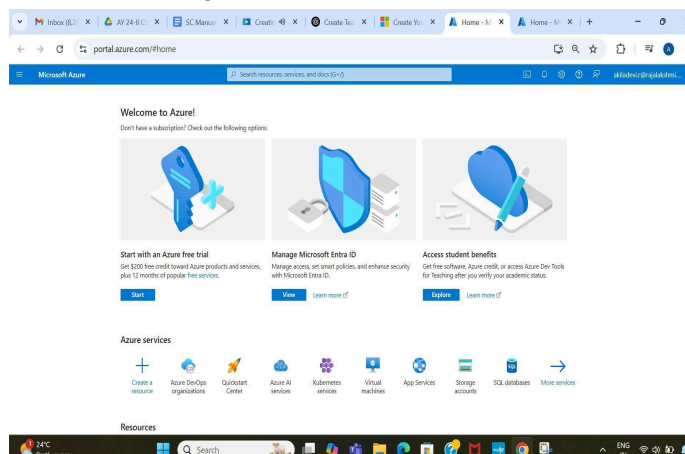
"As a [role], I [want to], [so that]."

Procedure:

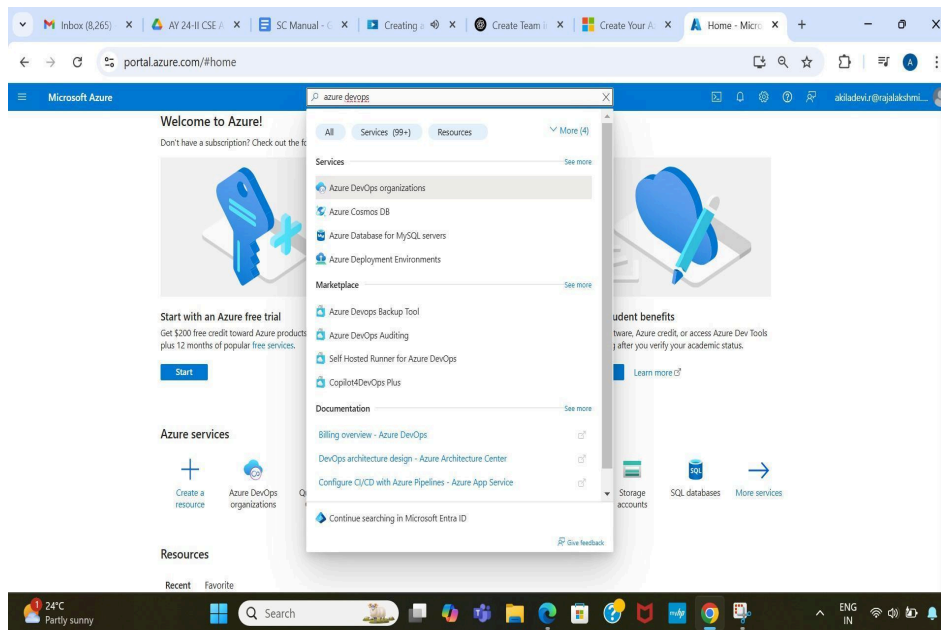
1. Open your web browser and go to the Azure website:
<https://azure.microsoft.com/en-in> Sign in using your Microsoft account credentials. If you don't have an account, you'll need to create one.
2. _____ If you don't have a Microsoft account, you can sign up for
<https://signup.live.com/?lic=1>



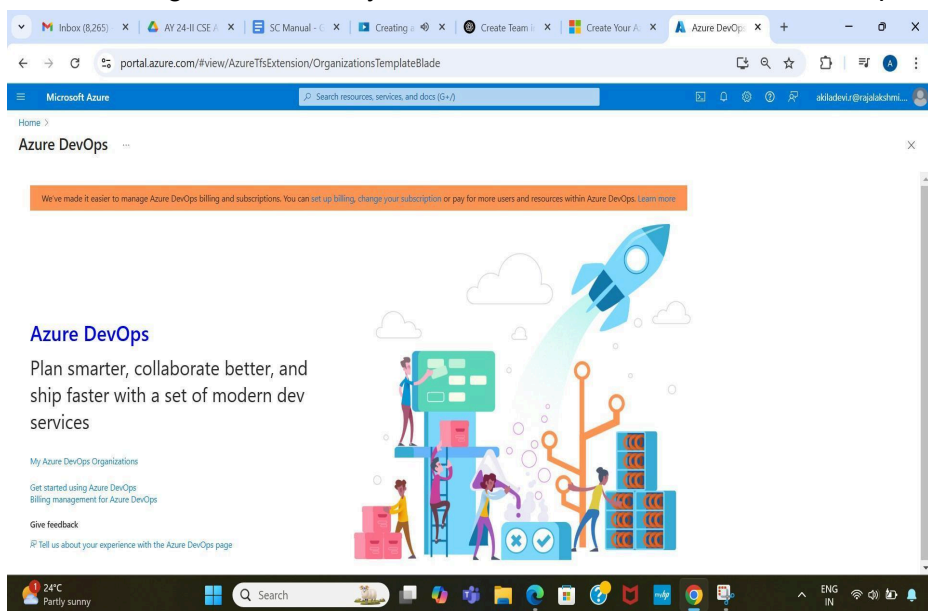
3. Azure home page



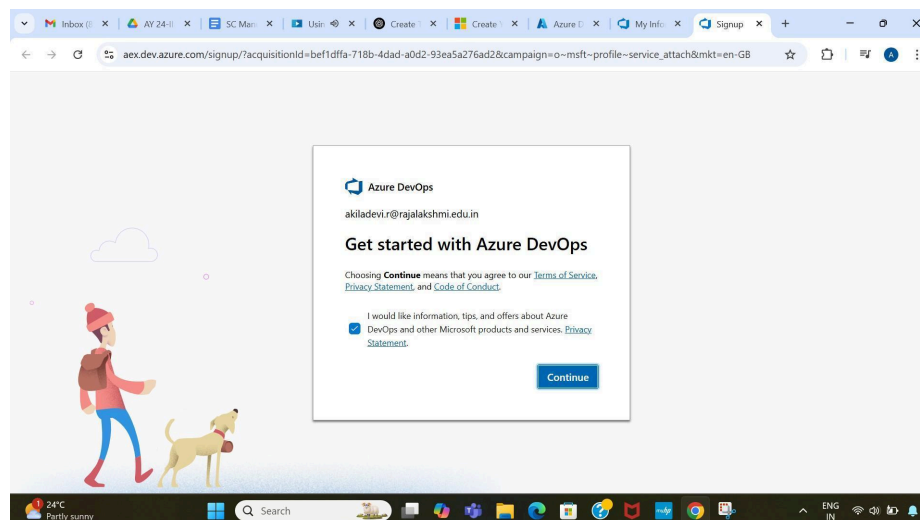
4. Open DevOps environment in the Azure platform by typing Azure DevOps Organizations in the search bar.

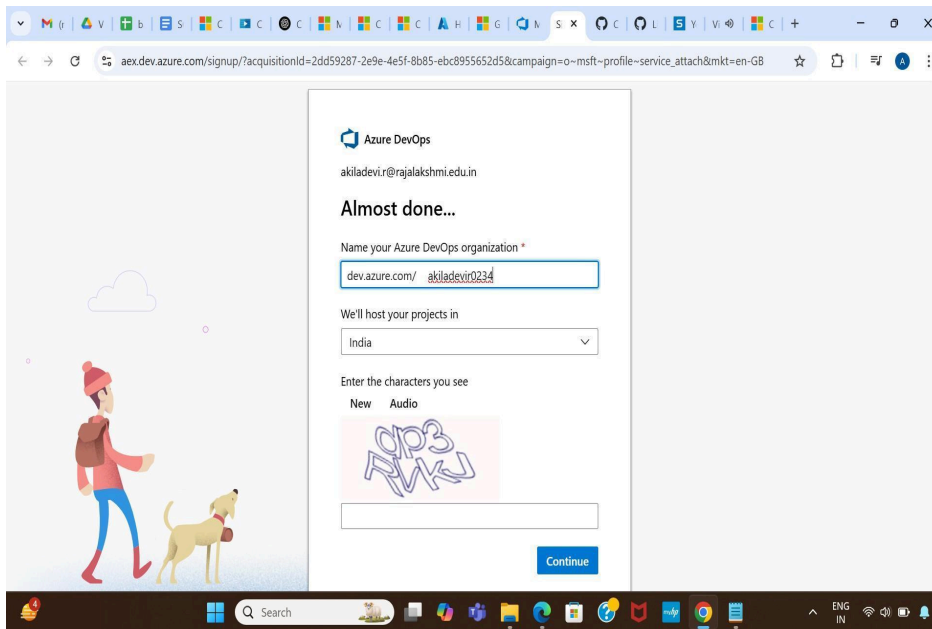


5. Click on the My Azure DevOps Organization link and create an organization and you should be taken to the Azure DevOps Organization



Home page.



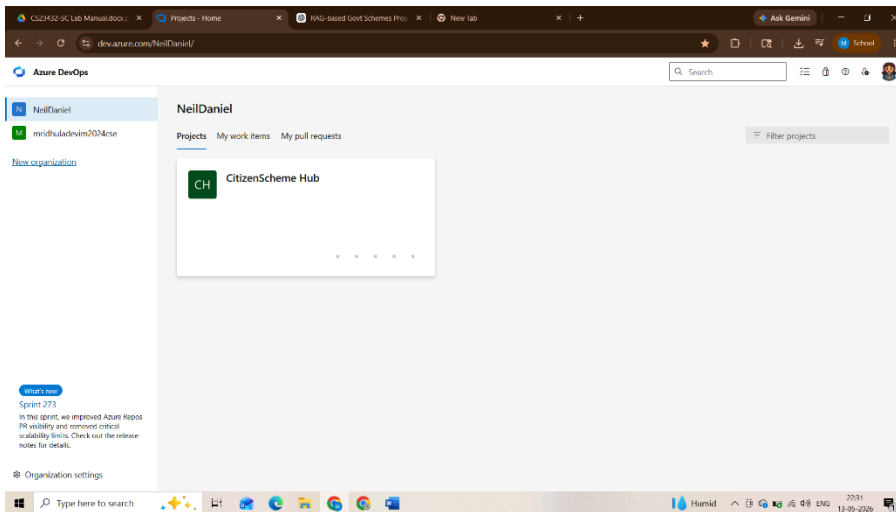


6. Create the First Project in Your Organization

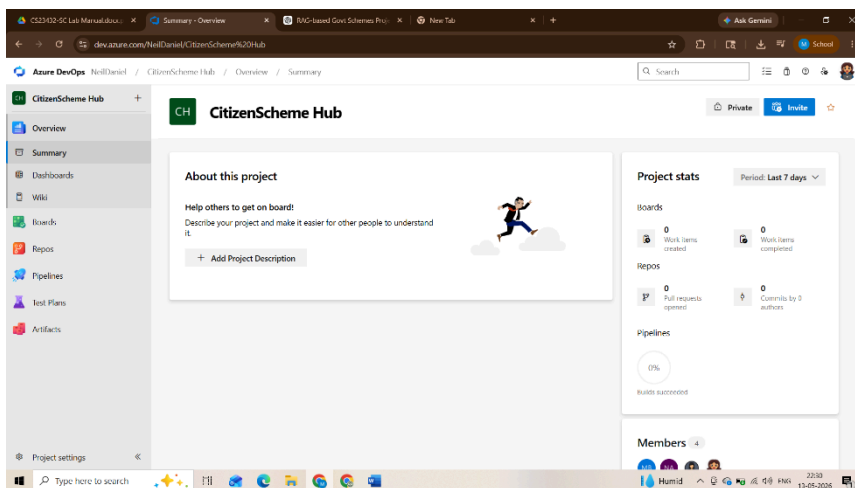
After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.

- i. On the organization's **Home page**, click on the **New Project** button.
- ii. Enter the project name, description, and visibility options:
 - o **Name:** Choose a name for the project (e.g., **LMS**).
 - o **Description:** Optionally, add a description to provide more context about the project.
 - o **Visibility:** Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).
- iii. Once you've filled out the details, click **Create** to set up your first project.

7. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.

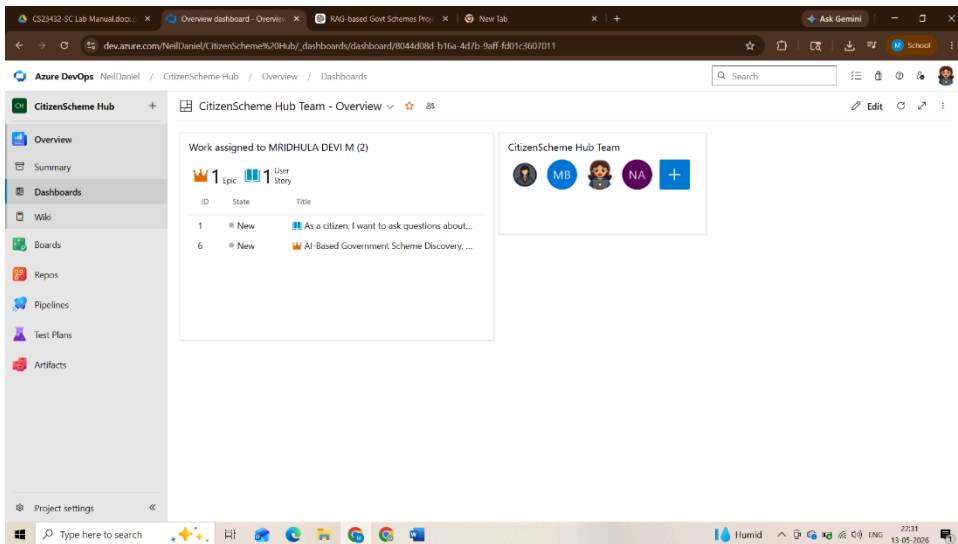


8. Project dashboard

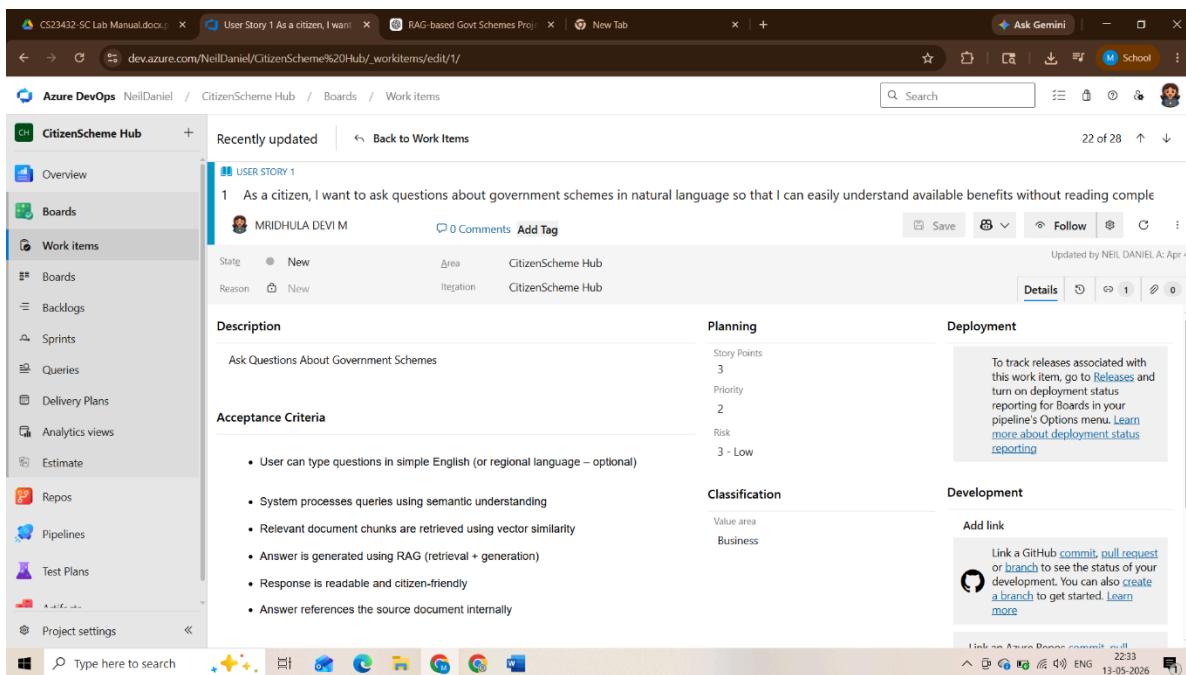


9. To manage user stories

- a. From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.
- b. On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a **+** button or **Add New Work Item** depending on the view you're in. From the **Add a work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.



10. Fill in User Story Details



Result:

The user story was written successfully.

EX NO. 5

SEQUENCE DIAGRAM

AIM:

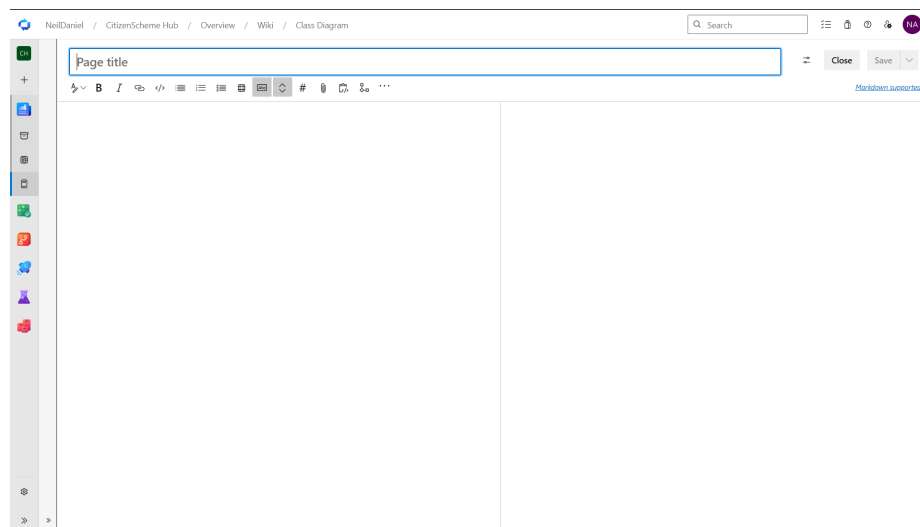
To design a Sequence Diagram by using Mermaid.js

THEORY:

A Sequence Diagram is a key component of Unified Modelling Language (UML) used to visualize the interaction between objects in a sequential order. It focuses on how objects communicate with each other over time, making it an essential tool for modelling dynamic behaviour in a system.

PROCEDURE:

1. Open a project in Azure DevOps Organisations.
2. To design select wiki from menu



3. Write code for drawing sequence diagram and save the code.

sequenceDiagram

participant U as User

participant F as Frontend (HTML/CSS/JS)

participant B as Spring Boot Backend

participant P as PDF Loader (PDFBox)

participant S as Search Service

participant D as PDF Documents

B->>P: loadAllPDFs()

P->>D: open and extractText()

D-->>P: raw text content

P-->>B: store in memory Map(filename, text)

U->>F: Enter search query

F->>B: search query sent

B->>S: search(query, documentMap)

S->>S: lowercase + keyword match + extract snippet

S-->>B: List of SearchResult(filename, snippet)

B-->>F: response generation

F-->>U: Display scheme name and snippet

Explanation:

participant defines the entities involved.

--> represents a direct message.

-->> represents a response message.

-> Solid line without arrow

--> Dotted line without arrow

-->> Solid line with arrowhead

-->> Dotted line with arrowhead

<<-->> Solid line with bidirectional arrowheads (v11.0.0+)

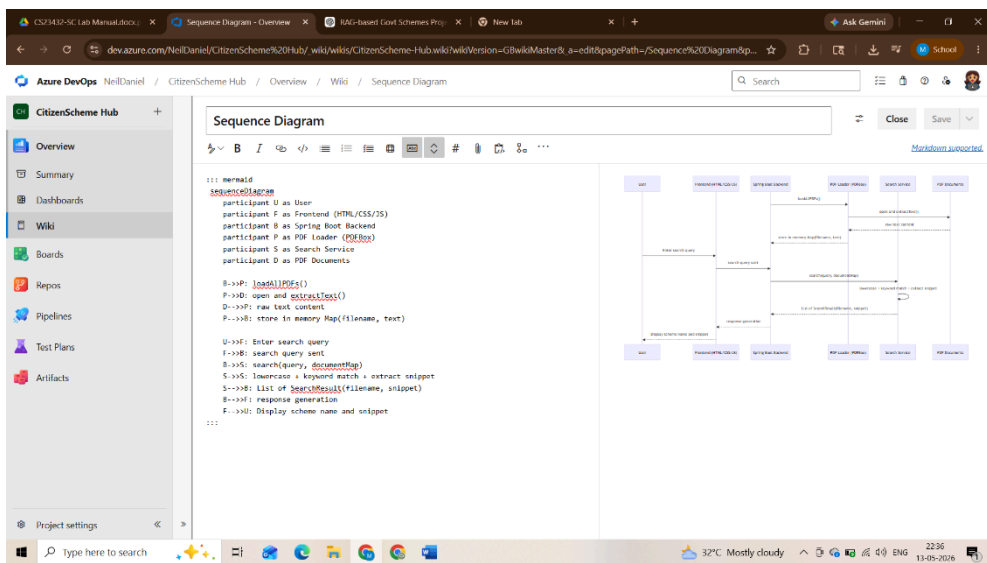
<<-->> Dotted line with bidirectional arrowheads (v11.0.0+)

-x Solid line with a cross at the end

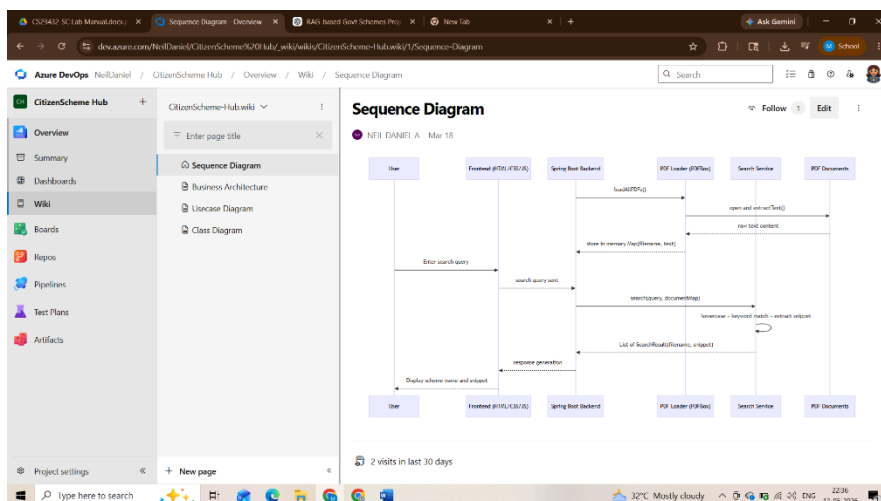
--x Dotted line with a cross at the end

-) Solid line with an open arrow at the end (async)

--) Dotted line with an open arrow at the end (async)



4. click wiki menu and select the page



RESULT:

The sequence diagram was drawn successfully.

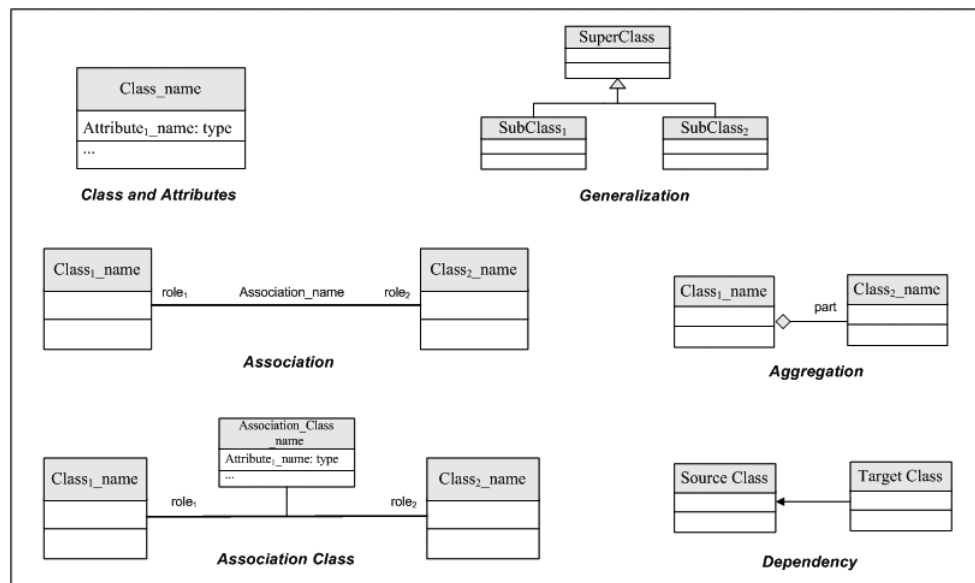
CLASS DIAGRAM

AIM :-

To draw a sample class diagram for your project or system.

THEORY

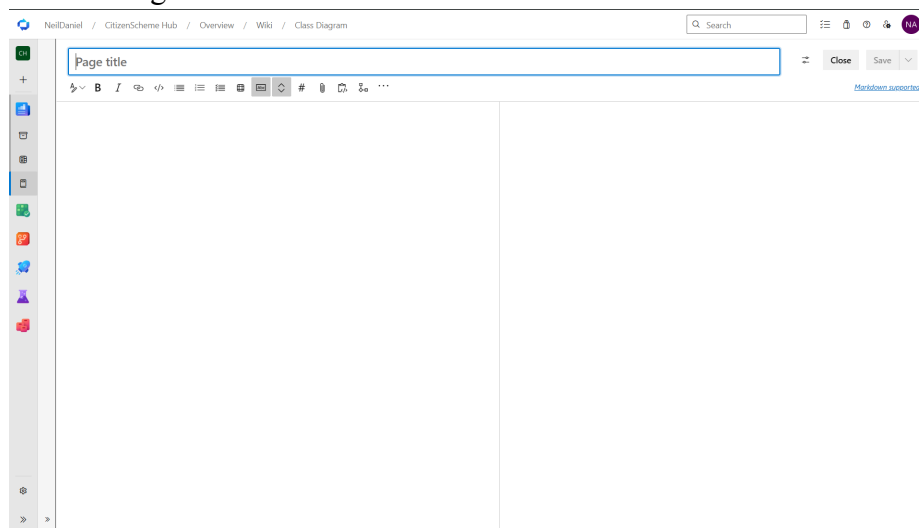
A UML class diagram is a visual tool that represents the structure of a system by showing its classes, attributes, methods, and the relationships between them.



Notations in class diagram

Procedure:

1. Open a project in Azure DevOps Organisations.
2. To design select wiki from menu



3. Write code for drawing class diagram and save the code

```

classDiagram
    class USER {
        +uid
        +name
        +email
        +login()
        +submitQuery()
        +viewResponse()
    }

    class Query {
        +queryText
        +queryTime
        +forawrdQuery()
    }

    class RAGService {
        +retrieves
        +retrieveDocuments()
    }

    class EmbeddingServices {
        +modelType
        +createEmbedding()
    }

    class Document {
        +docid
        +docT
        +content
        +splitcontent()
        +generateEmbedding()
    }

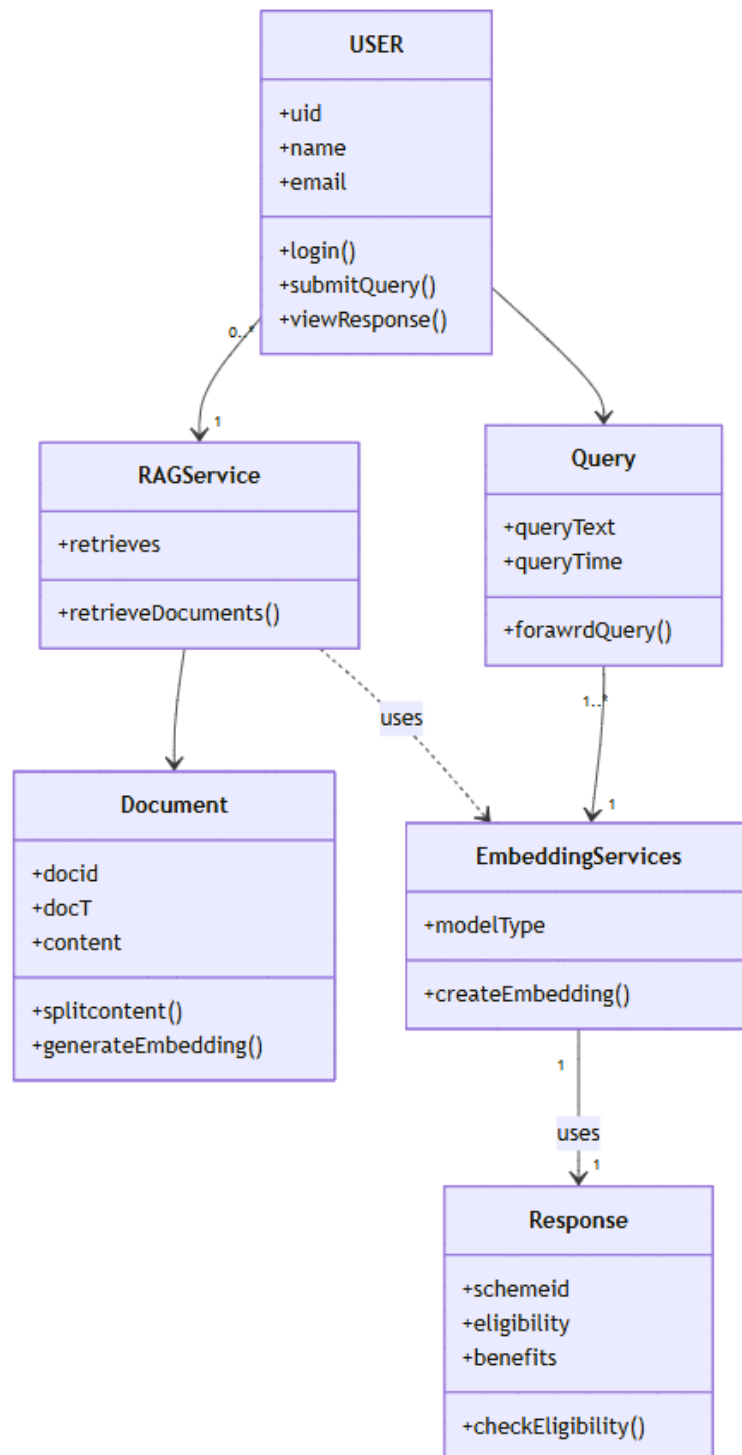
    class Response {
        +schemeid
        +eligibility
        +benefits
        +checkEligibility()
    }

    USER "0..*" --> "1" RAGService
    USER --> Query
    RAGService --> Document
    RAGService ..> EmbeddingServices : uses
    Query "1..*" --> "1" EmbeddingServices
    EmbeddingServices "1" --> "1" Response : uses

```

Relationship Types

Type	Description
<	Inheritance
*	Composition
o	Aggregation
>	Association
<	Association
>	Realization



Result:

The use case diagram was designed successfully.

USECASE DIAGRAM

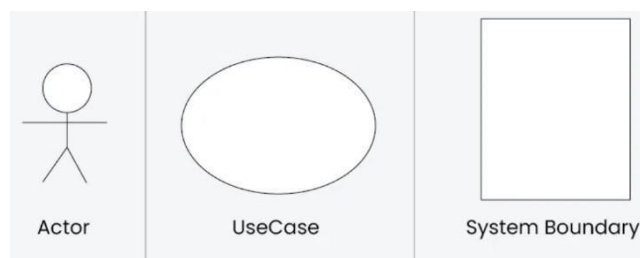
AIM :-

Steps to draw the Use Case Diagram using draw.io

THEORY:

- UCD shows the relationships among actors and use cases within a system which Provide an overview of all or part of the usage requirements for a system or organization in the form of an essential model or a business model and communicate the scope of a development project

- **Use Cases**
- **Actors**
- **Relationships**
- **System Boundary Boxes**



Procedure

Step 1: Create the Use Case Diagram in Draw.io

- Open Draw.io (draw.io).
- Click "Create New Diagram" and select "Blank" or "UML Use Case" template.
- Add Actors (Users, Admins, External Systems) from the UML section.
- Add Use Cases (Functionalities) using ellipses.
- Connect Actors to Use Cases with lines (solid for direct interaction, dashed for <<include>> and <<extend>>).
- Save the diagram as .drawio or export as PNG/JPG/SVG.

Step 2: Upload the Diagram to Azure DevOps

Option 1: Add to Azure DevOps Wiki

- Open Azure DevOps and go to your project.
- Navigate to Wiki (Project > Wiki).
- Click "Edit Page" or create a new page.
- Drag & Drop the exported PNG/JPG image.
- Use Markdown to embed the diagram:
- ![Use Case Diagram](attachments/use_case_diagram.png)

flowchart LR

User(("User"))

Admin(("Admin"))

subgraph System ["CitizenScheme Hub"]

UC1([Login / Register])

UC2([Search Government Scheme])

UC3([Ask Query])

UC4([Check Eligibility Criteria])

UC5([View Application Steps])

UC6([Upload Documents])

UC7([Manage Knowledge Base])

UC8([Update / Delete Scheme])

UC9([Manage Query Logs])

end

User --> UC1

User --> UC2

User --> UC3

User --> UC4

UC4 -.->|include| UC5

Admin --> UC6

Admin --> UC7

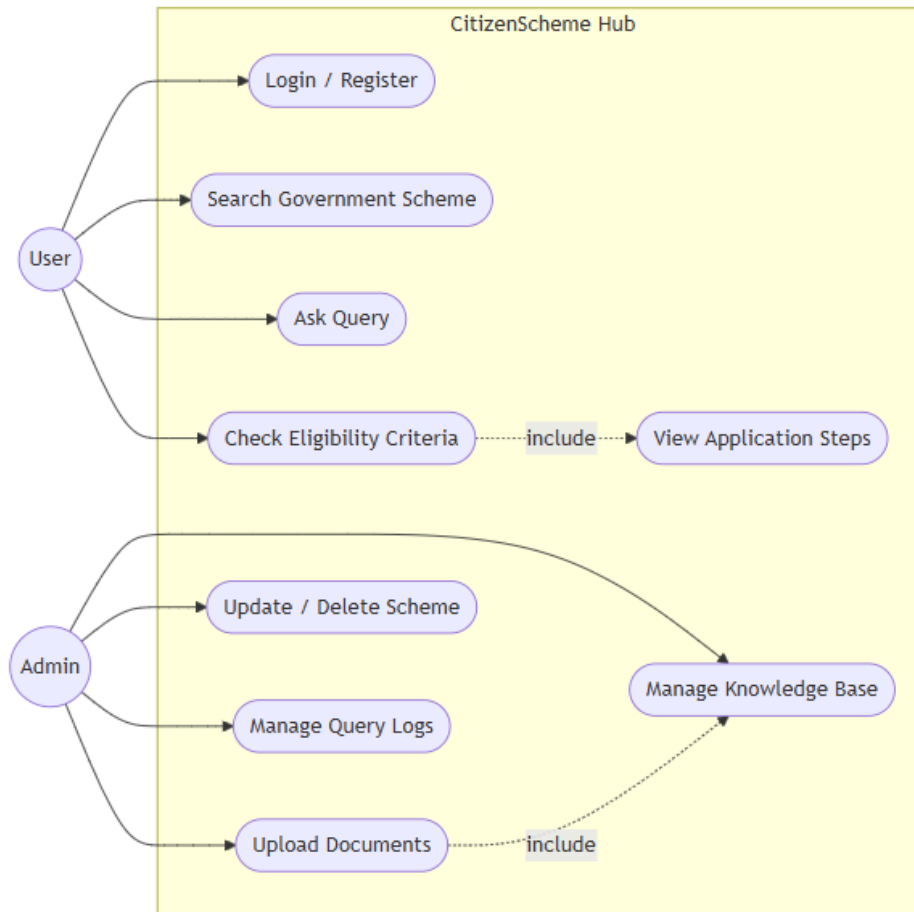
Admin --> UC8

Admin --> UC9

UC6 -.->|include| UC7

Option 2: Attach to Work Items in Azure Boards

- Open Azure DevOps → Navigate to Boards (Project > Boards).
- Select a User Story, Task, or Feature.
- Click "Attachments" → Upload your Use Case Diagram.
- Add comments or descriptions to explain the use case.



Result:

The use case diagram was designed successfully

EX NO. 8

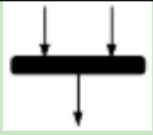
ACTIVITY DIAGRAM

AIM :-

To draw a sample activity diagram for your project or system.

THEORY

Activity diagrams are an essential part of the Unified Modelling Language (UML) that help visualize workflows, processes, or activities within a system. They depict how different actions are connected and how a system moves from one state to another.

Notations	Symbol	Meaning
Start		Shows the beginning of a process
Connector		Shows the directional flow, or control flow, of the activity
Joint symbol		Combines two concurrent activities and re-introduces them to a flow where one activity occurs at a time
Decision		Represents a decision
Note		Allows the diagram creators to communicate additional messages
Send signal		Show that a signal is being sent to a receiving activity
Receive signal		Demonstrates the acceptance of an event
Flow final symbol		Represents the end of a specific process flow
Option loop		Allows the creator to model a repetitive sequence within the option loop symbol
Shallow history pseudostate		Represents a transition that invokes the last active state.
End		Marks the end state of an activity and represents the completion of all flows of a process

Procedure

1. Draw diagram in draw.io
2. Upload the diagram in Azure DevOps wiki

Result:

The activity diagram was designed successfully

EX NO. 9

ARCHITECTURE DIAGRAM

AIM:

Steps to draw the Architecture Diagram using draw.io.

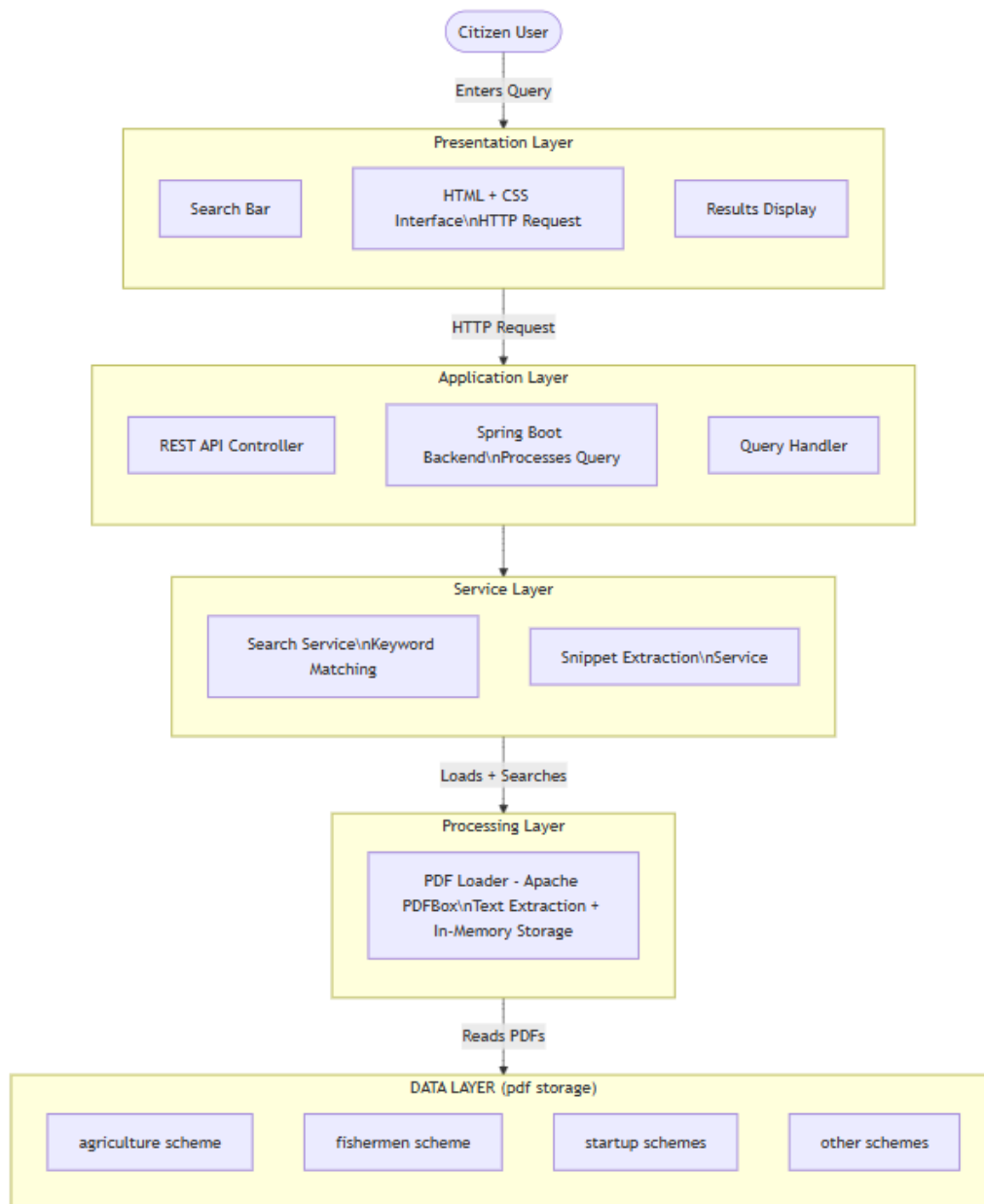
THEORY:

An architectural diagram is a visual representation that maps out the physical implementation for components of a software system. It shows the general structure of the software system and the associations, limitations, and boundaries between each element.



PROCEDURE

1. Draw diagram in draw.io



2. Upload the diagram in Azure DevOps wiki or use the mermaid code

```
graph TD
    CU([Citizen User]) -->|Enters Query| PL
    subgraph PL ["Presentation Layer"]
        SB[Search Bar]
        HI[HTML + CSS Interface\nHTTP Request]
        RD[Results Display]
    end
    PL -->|HTTP Request| AL
    subgraph AL ["Application Layer"]
        RA[REST API Controller]
        SB2[Spring Boot Backend\nProcesses Query]
        QH[Query Handler]
    end
    AL --> SL
    subgraph SL ["Service Layer"]
        SS[Search Service\nKeyword Matching]
        SE[Snippet Extraction\nService]
    end
    SL -->|Loads + Searches| PRL
    subgraph PRL ["Processing Layer"]
        PD[PDF Loader - Apache PDFBox\nText Extraction + In-Memory Storage]
    end
    PRL -->|Reads PDFs| DL
    subgraph DL ["DATA LAYER (pdf storage)"]
        AG[agriculture scheme]
        FI[fishermen scheme]
        ST[startup schemes]
        OT[other schemes]
    end
```

RESULT:

The architecture diagram was designed successfully

USER INTERFACE

AIM:

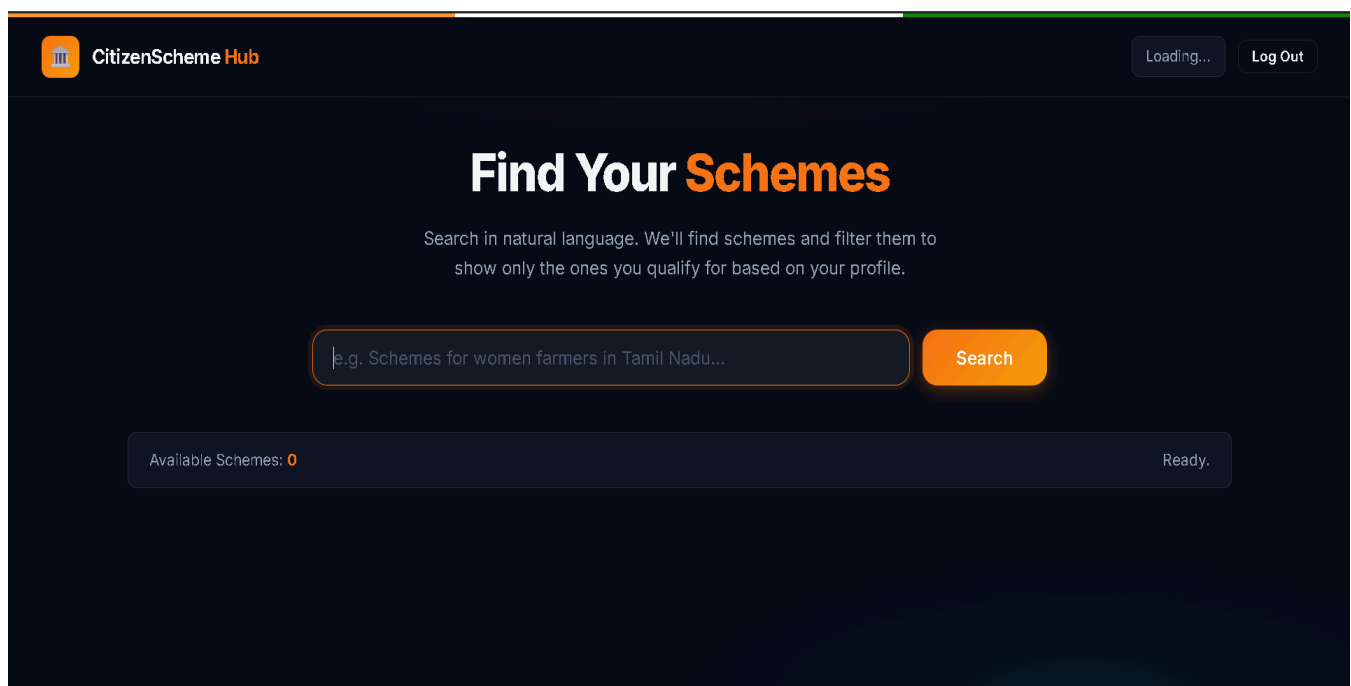
Design User Interface for the given project

THEORY:

A web application frontend is the part of a software that users directly see and interact with. It is built using HTML for structure, CSS for styling, and JavaScript for interactivity. Static Web Apps are websites made of plain HTML, CSS, and JavaScript files that do not need a server to process pages. Microsoft Azure Static Web Apps is a cloud service that hosts such static websites and makes them publicly accessible on the internet. It automatically deploys the app from a GitHub repository and provides a free public URL with SSL certificate.

The CitizenScheme Hub frontend consists of three pages:

- index.html — The main scheme discovery and search page
- login.html — User authentication page
- signup.html — New user registration with eligibility profile



INDEX PAGE



CitizenScheme Hub

Sign in to find schemes you qualify for

EMAIL ADDRESS

PASSWORD

Sign In

Don't have an account? [Create one](#)

SIGN IN PAGE



Create Your Profile

Tell us about yourself to find schemes you're eligible for.

EMAIL ADDRESS

PASSWORD (MIN 6 CHARACTERS)

Eligibility Details

AGE

ANNUAL INCOME (₹)

GENDER

Male

SOCIAL CATEGORY (CASTE)

All

STATE / UT

OCCUPATION (OPTIONAL)

Complete Registration

Already have an account? [Sign In](#)

SIGN UP PAGE

PROCEDURE

- Open dev.azure.com and select the CitizenScheme Hub project.
- Click Repos from the left menu and copy the Clone URL by clicking the Clone button.
- Open terminal in VS Code and initialize git in your project folder:
`git init`
- Add the Azure DevOps remote URL:
`git remote add origin <paste-clone-url-here>`
- Stage all the project files:
`git add.`
- Commit the files:
`git commit -m "Added project files"`
- Push the code to Azure Repos:
`git push -u origin main`
- Go back to Azure DevOps → Repos → Files to verify all files are uploaded successfully.

Result:

The UI was designed successfully.

Aim:

To implement the given project based on Agile Methodology.

Procedure:**Step 1: Set Up an Azure DevOps Project**

- Log in to Azure DevOps.
- Click "New Project" → Enter project name → Click "Create".
- Inside the project, navigate to "Repos" to store the code.

Step 2: Add Your Web Application Code

- Navigate to Repos → Click "Clone" to get the Git URL.
- Open Visual Studio Code / Terminal
and run: `git clone <repo_url>`
`cd <repo_folder>`
- Add web application code (HTML, CSS, JavaScript, React, Angular, or backend like Node.js, .NET, Python, etc.).
- Commit & push:
`git add .`
`git commit -m "Initial commit"`
`git push origin main`

Step 3: Set Up Build Pipeline (CI/CD - Continuous Integration)

- Navigate to Pipelines → Click "New Pipeline".
- Select Git Repository (Azure Repos, GitHub, or Bitbucket).
- Choose Starter Pipeline or a pre-configured template for your framework.
- Modify the azure-pipelines.yml file (Example for a Node.js app):

```
trigger:
- main

pool:
  vmImage: 'ubuntu-latest'

steps:
- task: UseNode@1
  inputs:
    version: '16.x'

- script: npm install
  displayName: 'Install dependencies'

- script: npm run build
  displayName: 'Build application'

- task:
  PublishBuildArtifacts@1
  inputs:
    pathToPublish: 'dist'
```

artifactName: 'drop'

Click "Save and Run" → The pipeline will start building app.

Step 4: Set Up Release Pipeline (CD - Continuous Deployment)

- Go to Releases → Click "New Release Pipeline".
- Select Azure App Service or Virtual Machines (VMs) for deployment.
- Add an artifact (from the build pipeline).
- Configure deployment stages (Dev, QA, Production).
- Click "Deploy" to push your web app to Azure.

Result

Thus the application was successfully implemented.